LLM2Seq: Repurposing LLM Source Encoders for Encoder-Decoder Summarization with Self-Distilled Multi-Token Prediction

Giang-Bien Kieu
Hanoi University of Science and Technology
HUST, Vietnam
bienkieugiang@gmail.com

Xuan-Dung Doan
Viettel Artificial Intelligence Center
Viettel Group, Vietnam
dungdx4@viettel.com.vn

Abstract

Abstractive summarization is a sequence-to-sequence task. The model must read a document, keep the important evidence, and then write a shorter text. Modern LLM checkpoints are strong generators and representation models, but many of them do not directly expose an encoder-decoder interface for summarization. Encoder-decoder models fit this workflow more naturally, yet they usually require a native encoder trained in that form. This report presents LLM2Seq, a framework that repurposes pretrained LLM source encoders inside an encoder-decoder summarizer. A gated residual adapter turns its source states into cross-attention memory for a lightweight Transformer decoder. The encoder reads the document, while the decoder writes the summary. We also study inference speed with self-distilled Multi-Token Prediction (MTP). After the summarizer is trained, MTP heads learn from the frozen decoder, draft future tokens, and are verified during decoding. We evaluate LLM2Seq on Vietnamese WikiLingua. The results show that LLM-based source encoders can serve as useful cross-attention memory for summarization, and that MTP speedups must be judged by real end-to-end throughput rather than accepted tokens alone.

Keywords: abstractive summarization; encoder-decoder models; LLM source encoders; adapters; multi-token prediction; Vietnamese WikiLingua

1 Introduction

1.1 Motivation

Abstractive summarization naturally follows a sequence-to-sequence formulation. A model reads a document, keeps the important evidence, and writes a shorter text. Encoder-decoder models match this shape because the encoder reads the source and the decoder writes the target (Sutskever et al., 2014; Bahdanau et al., 2015; Vaswani et al., 2017). Decoder-only LLMs are different. They are strong generators, but they mix the source and target into one left-to-right sequence (Brown et al., 2020; Dubey et al., 2024; Yang et al., 2024). This makes them easy to prompt, but it does not explicitly separate source understanding from summary generation.

1.2 Problem Statement

This report asks whether pretrained LLM-based source encoders can be reused inside an encoder-decoder summarizer. The task is Vietnamese abstractive summarization on WikiLingua. Given a source document x, the model must produce a summary y. The desired system should keep the representation strength of modern LLMs, use an encoder-decoder interface for summarization, and decode faster when possible.

1.3 Gap with Existing Work

Pretrained encoder-decoder models such as T5, BART, and mT5 remain strong summarization baselines (Raffel et al., 2020; Lewis et al., 2020; Xue et al., 2021). However, they are not a direct way to reuse representations from recent LLM checkpoints. Recent work shows that decoder-only models can serve as encoders (BehnamGhader et al., 2024; Luo et al., 2025), but summarization still needs a bridge from source-side LLM states to decoder cross-attention. Context compression models such as LCLMs compress long prompts into shorter latent sequences (Li et al., 2026). That is a related but different goal. Our goal is not general prompt compression. Our goal is task-specific summarization with token-level source memory.

1.4 Contributions

We propose **LLM2Seq**, an encoder-decoder summarization framework built from repurposed LLM source encoders. The contributions are:

- We repurpose LLM-based source encoders for Vietnamese abstractive summarization, including a decoder-only-derived LLM2Vec encoder and a Qwen embedding encoder.
- We introduce a gated residual memory adapter that turns LLM hidden states into cross-attention memory for a lightweight decoder.
- We train self-distilled MTP heads after the summarizer is trained, keeping the main encoder-adapter-decoder path frozen.
- We show that MTP must be evaluated by real throughput, because accepted tokens do not always become wall-clock speed.

1.5 Report Organization

Section 2 reviews encoder-decoder summarization, decoder-only LLM encoders, context compression, and efficient decoding. Section 3 presents LLM2Seq and the MTP decoding method. Section 4 describes the experimental setup. Section 5 reports the results and analysis. Sections 6–9 discuss the demo, validity, limitations, and conclusion.

2 Background and Related Work

2.1 Encoder-Decoder Summarization

Encoder-decoder models separate reading from writing. The encoder maps the input sequence into contextual states. The decoder generates the output by attending to these states. Attention-based sequence-to-sequence models introduced this interface (Sutskever et al., 2014; Bahdanau et al., 2015), and the Transformer made self-attention and cross-attention standard (Vaswani et al., 2017). T5, BART, and mT5 show that this design works well for summarization (Raffel et al., 2020; Lewis et al., 2020; Xue et al., 2021).

2.2 Decoder-only LLMs as Encoders

Decoder-only LLMs are usually trained with next-token prediction. Their causal mask is natural for generation but not for bidirectional source encoding. LLM2Vec shows that decoder-only LLMs can be converted into strong text encoders (BehnamGhader et al., 2024). GRIT studies generative models as representation learners (Muennighoff et al., 2024). LaMaTE uses LLM encoders for machine translation (Luo et al., 2025). T5Gemma explores encoder-decoder models built from modern decoder-only components (Zhang et al., 2025). LLM2Seq follows this line, but focuses on summarization. The Llama variant is the cleaner decoder-only-derived encoder case, while the Qwen variant should be read as an embedding-oriented LLM source encoder.

2.3 Context Compression and LCLM

End-to-End Context Compression at Scale introduces LCLMs, which compress long contexts into shorter latent sequences for general long-context inference (Li et al., 2026). LCLMs optimize compression ratio, memory, time-to-first-token, and long-context accuracy. LLM2Seq uses an encoder-decoder memory interface, but it does not compress the source with a fixed ratio. It keeps token-level memory because summaries often depend on local source evidence.

2.4 Efficient Decoding and Multi-Token Prediction

Autoregressive decoding emits one token per step. Speculative decoding drafts tokens and verifies them with a target model (Leviathan et al., 2023). Blockwise decoding predicts several future tokens in parallel (Stern et al., 2018). Medusa adds extra decoding heads (Cai et al., 2024). Multi-token prediction trains heads to predict future tokens directly (Gloeckle et al., 2024). MTP-D adds self-distillation to improve these heads (Zhao et al., 2026), following knowledge distillation (Hinton et al., 2015).

2.5 Positioning of LLM2Seq

LLM2Seq sits between encoder-decoder summarization and LLM source-encoder reuse. It is not a general context compressor. It is also not a pure decoder-only prompting method. It asks a narrower question: can source-side LLM representations be adapted into useful cross-attention memory for summarization, and can verified MTP accelerate that decoder in practice? Table 1 summarizes this position against end-to-end context compression.

3 LLM2Seq Method

3.1 Problem Formulation

Let $x = (x_1, \ldots, x_m)$ be the source document and $y = (y_1, \ldots, y_n)$ be the target summary. Let $a_i^x \in \{0, 1\}$ and $a_t^y \in \{0, 1\}$ be the source and target

Aspect	LCLM / End-to-End Context Compression	LLM2Seq
Main goal	Compress long context for general inference	Vietnamese abstractive summarization
Source representation	Short latent compressed sequence	Token-level cross-attention memory
Compression ratio	Explicit optimization target	Not the main objective
Output setting	General long-context inference	Encoder-decoder summary generation
Speed mechanism	Shorter context and reduced memory	Verified MTP decoding
Risk	Information loss from compression	More memory and cross-attention cost
Best use case	Long redundant context	Source-grounded summarization

Table 1: Conceptual positioning of LLM2Seq against LCLM-style end-to-end context compression. The comparison is about modeling goals and inference mechanisms, not an empirical LCLM baseline.

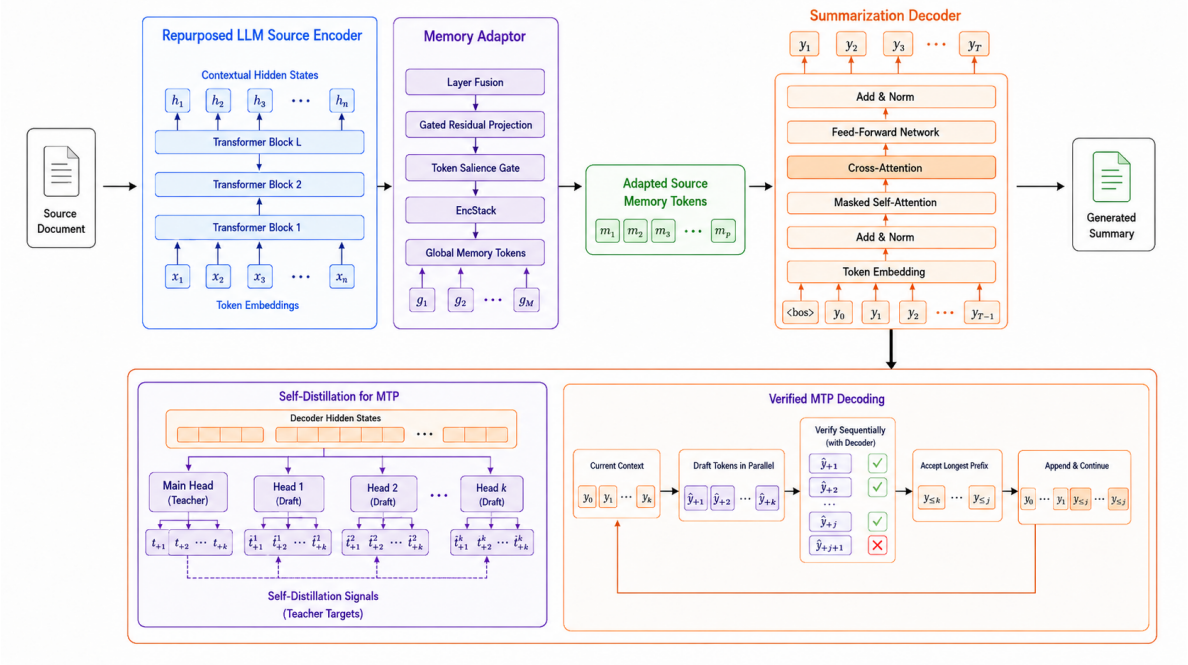

Figure 1: LLM2Seq repurposes a pretrained LLM source encoder, maps its hidden states through a gated residual memory adapter, and uses a lightweight cross-attentive decoder for summarization. The lower blocks show self-distilled MTP training and verified MTP decoding.

masks. The model estimates:

$$p(y|x) = \prod_{t=1}^{n} p(y_t \mid y_{<t}, x).$$

LLM2Seq makes the conditioning path explicit. The source encoder builds token states:

$$H = E_\theta(x), \qquad H \in \mathbb{R}^{m \times d_e}.$$

The adapter maps them into decoder memory:

$$M = A_\phi(H), \qquad M \in \mathbb{R}^{(m+g) \times d_d},$$

where g is the number of learned global memory tokens. The decoder then generates:

$$p(y|x) = \prod_{t=1}^{n} p_\psi(y_t \mid y_{<t}, M).$$

Training uses teacher forcing. With non-padding target positions $\Omega = \{t : a_t^y = 1\}$, the summarization loss is:

$$\mathcal{L}_{sum} = -\frac{1}{|\Omega|} \sum_{t \in \Omega} \log p_\psi(y_t \mid y_{<t}, M).$$

This form is the core design choice. The encoder handles source understanding. The decoder handles target generation. The adapter decides how source evidence is exposed to cross-attention.

3.2 Overall Architecture

Figure 1 shows the full system. LLM2Seq has three main parts. First, a pretrained LLM-based source encoder is used; our instantiations are a decoder-only-derived LLM2Vec encoder and a Qwen embedding encoder. Second, a gated residual memory adapter reshapes the encoder

states into decoder memory. Third, a lightweight Transformer decoder generates the summary with masked self-attention and cross-attention. After the summarizer is trained, an MTP module is attached for acceleration. The MTP module is trained separately and verified by the main decoder during inference.

3.3 Repurposed LLM Source Encoder

We use two source encoders. The Llama variant uses an LLM2Vec Sheared-LLaMA checkpoint. The Qwen variant uses Qwen3 Embedding 0.6B. This distinction matters for the scope of the claim: LLM2Seq studies reusable LLM source encoders, not only pure decoder-only checkpoints. For a source sequence x, the encoder returns token-level hidden states:

$$H^{(\ell)} = F_\theta^{(\ell)}(x, a^x), \qquad H^{(\ell)} \in \mathbb{R}^{m \times d_e}.$$

Unlike sentence embedding use cases, LLM2Seq keeps the full sequence of token states. This is needed because the decoder must attend to local source evidence. The encoder also returns a set of selected layers:

$$\mathcal{H}_S = \{H^{(l_1)}, H^{(l_2)}, \ldots, H^{(l_s)}\}.$$

In our setting, the selected layers are $[-1, -4, -8, -12]$. Full fine-tuning is expensive. Phase 2 therefore applies LoRA to the encoder (Hu et al., 2022). For a frozen encoder weight matrix W, LoRA learns a low-rank update:

$$W' = W + \frac{\alpha}{r} BA,$$

where $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. Only the low-rank matrices are updated for the encoder. This adapts the source representation to Vietnamese summarization while keeping training practical.

3.4 Gated Residual Memory Adapter

The adapter is the bridge between the LLM encoder and the decoder. It must solve a mismatch. The encoder states live in the LLM representation space, while the decoder expects memory in its own hidden space. The adapter also needs to keep token-level evidence, because summaries often depend on details in the source.

First, token-wise layer fusion combines selected encoder layers. For token position i and selected layer j, a score is computed by:

$$e_{ij} = w_f^\top H_i^{(l_j)}.$$

The layer mixture is:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{r=1}^{s} \exp(e_{ir})}, \qquad \bar{H}_i = \sum_{j=1}^{s} \alpha_{ij} H_i^{(l_j)}.$$

This lets different tokens use different depths of the LLM. Lexical cues and semantic cues do not always appear at the same layer.

The fused states are then projected into decoder space. The implemented gated residual projection first normalizes the fused state:

$$\hat{H} = \mathrm{LN}(\bar{H}).$$

It builds a base path, a gate, and an update:

$$P = W_p \hat{H} + b_p, \qquad G = \sigma(W_g \hat{H} + b_g),$$

$$U = W_2\, \mathrm{GELU}(W_1\, \mathrm{LN}(P) + b_1) + b_2.$$

The adapted token state is:

$$R = P + G \odot U.$$

The base path preserves information after dimensionality change. The gated update adds task-specific changes only where useful.

A token salience gate then estimates which source positions should be emphasized:

$$S = \sigma(W_{s2}\, \mathrm{GELU}(W_{s1}\, \mathrm{LN}(R) + b_{s1}) + b_{s2}).$$

The implementation does not delete low-salience tokens. It rescales them:

$$R'_i = (0.5 + S_i) R_i.$$

Padding positions have $S_i = 0$. Thus padded memory cannot become salient.

An EncStack refines the memory with $L_a = 4$ pre-norm Transformer encoder layers:

$$\bar{R}^{(0)} = R',$$

$$Q^{(u)} = \bar{R}^{(u-1)} + \mathrm{SelfAttn}(\mathrm{LN}(\bar{R}^{(u-1)}), a^x),$$

$$\bar{R}^{(u)} = Q^{(u)} + \mathrm{FFN}(\mathrm{LN}(Q^{(u)})).$$

Let $\tilde{R} = \bar{R}^{(L_a)}$. The final memory concatenates learned global memory tokens G_m with the refined token memory:

$$M = [G_m;\ \tilde{R}].$$

The corresponding memory mask is:

$$a^M = [\mathbf{1}_g;\ a^x].$$

The global tokens give the decoder a small document-level workspace. The token memory gives the decoder direct access to local evidence.

3.5 Lightweight Cross-Attentive Decoder

The decoder is an 8-layer Transformer decoder with hidden size 1024, 16 attention heads, and a 4096-dimensional feed-forward layer. At step t, it reads the previous target tokens $y_{<t}$ and the adapted source memory M. The decoder embeds target tokens with a tied input/output embedding matrix E:

$$Y^{(0)} = E[y_{<t}].$$

It uses RoPE in causal self-attention. For a query-key pair (q_i, k_j), RoPE rotates each head by position-dependent matrices:

$$\tilde{q}_i = R_i q_i, \qquad \tilde{k}_j = R_j k_j.$$

Attention with mask B is:

$$\text{Attn}(Q, K, V, B) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + B\right)V.$$

The causal mask has $B_{ij} = -\infty$ for $j > i$. The cross-attention mask uses a^M so padded source tokens are ignored.

For decoder layer r, the pre-norm residual block is:

$$\tilde{Y}^{(r-1)} = \text{RMSNorm}(Y^{(r-1)}),$$

$$A^{(r)} = Y^{(r-1)} + \text{SelfAttn}(\tilde{Y}^{(r-1)}, B_{causal}),$$

$$\tilde{A}^{(r)} = \text{RMSNorm}(A^{(r)}),$$

$$C^{(r)} = A^{(r)} + \text{CrossAttn}(\tilde{A}^{(r)}, M, a^M),$$

$$Y^{(r)} = C^{(r)} + \text{SwiGLU}(\text{RMSNorm}(C^{(r)})).$$

The next-token distribution is:

$$o_t = E^\top \text{RMSNorm}(Y_t^{(L_d)}),$$

$$p_\psi(y_t \mid y_{<t}, M) = \text{softmax}(o_t).$$

The decoder is intentionally smaller than the encoder. This keeps generation cheaper while still allowing the encoder to carry strong source representations.

3.6 Self-Distilled Multi-Token Prediction

After Phase 2, the main summarizer is frozen. Phase 3 adds $K = 4$ cascaded MTP heads. Let $h_t^{(0)} = Y_t^{(L_d)}$ be the frozen decoder state at target position t. The main head predicts:

$$p_t^{(0)} = \text{softmax}(E^\top h_t^{(0)}).$$

Each MTP depth predicts one future token. During training, depth k receives the previous MTP hidden state and the teacher-forced embedding of y_{t+k-1}:

$$z_t^{(k)} = \text{LN}\left(W_c[h_t^{(k-1)}; E(y_{t+k-1})]\right).$$

Then a small causal block produces:

$$\bar{z}_t^{(k)} = z_t^{(k)} + \text{SelfAttn}(\text{LN}(z_{\le t}^{(k)})),$$

$$h_t^{(k)} = \bar{z}_t^{(k)} + \text{FFN}(\text{LN}(\bar{z}_t^{(k)})).$$

The MTP distribution is:

$$p_t^{(k)} = \text{softmax}(E^\top h_t^{(k)}), \qquad k = 1, \ldots, K.$$

Thus head k predicts y_{t+k}.

The hard-label loss trains the heads to predict the gold future token:

$$\ell_k^{CE} = -\frac{1}{|\Omega_k|}\sum_{t\in\Omega_k} \log p_t^{(k)}(y_{t+k}),$$

$$\mathcal{L}_{CE} = \frac{1}{\sum_k w_k}\sum_{k=1}^{K} w_k \ell_k^{CE},$$

where Ω_k contains valid non-padding positions for offset k. The self-distillation loss makes each MTP head follow the frozen main decoder at the same future position. Let $z_{t+k}^{(0)}$ be the detached main-head logits at position $t + k$. We keep the top-J vocabulary indices $\mathcal{V}_{t,k}$ from this teacher distribution. With temperature τ, the teacher and student distributions on this top-J set are:

$$q_{t,k} = \text{softmax}(z_{t+k,\mathcal{V}_{t,k}}^{(0)}/\tau),$$

$$s_{t,k} = \text{softmax}(z_{t,\mathcal{V}_{t,k}}^{(k)}/\tau).$$

The KL loss is:

$$\mathcal{L}_{KL} = \frac{\tau^2}{\sum_k w_k}\sum_{k=1}^{K} w_k \frac{1}{|\Omega_k|}\sum_{t\in\Omega_k} \text{KL}(q_{t,k} \,\|\, s_{t,k}).$$

The final Phase 3 loss is:

$$\mathcal{L}_{MTP} = \lambda_{CE}\mathcal{L}_{CE} + \gamma(u)\lambda_{KL}\mathcal{L}_{KL},$$

where u is the training progress. The ramp $\gamma(u)$ is 0 before 20% of training, increases linearly until 50%, and is 1 afterwards. We use $\lambda_{CE} = 0.6$, $\lambda_{KL} = 0.2$, $\tau = 1.5$, $J = 2048$, and head weights $[1.0, 0.8, 0.5, 0.25]$. Only the MTP heads are updated in this phase.

3.7 Verified MTP Decoding Algorithm

At inference time, the MTP heads draft future tokens. The main decoder verifies them. This keeps the output tied to the trained summarizer. Let $g(\pi)$ be the constrained greedy main-head token for prefix π. It includes the same repetition penalty, minimum length rule, and no-repeat trigram constraint as standard decoding. For a current prefix π, the main decoder first predicts:

$$c_0 = g(\pi).$$

The cascaded MTP module then drafts:

$$c_k = \arg\max_v p^{(k)}(v \mid \pi, c_{<k}),$$

$$k = 1, \ldots, K.$$

The candidate block is $c = (c_0, c_1, \ldots, c_K)$. The verifier runs the frozen decoder on the candidate block and computes:

$$v_j = g(\pi, c_0, \ldots, c_j), \qquad j = 0, \ldots, K.$$

The number of accepted draft tokens is:

$$a = \max\{r : c_j = v_{j-1} \text{ for all } 1 \le j \le r\}.$$

The emitted block is:

$$b = (c_0, \ldots, c_a, v_a),$$

unless an end token or maximum length cuts the block earlier. This design is intended to follow the constrained main greedy path. Accepted draft tokens reduce future decoding work, while v_a corrects the first mismatch.

The full procedure is:

1. Encode the source once and cache cross-attention keys and values.
2. Predict the main token c_0 from the current prefix.
3. Draft K future tokens with the cascaded MTP heads.
4. Verify the candidate block with the frozen main decoder.
5. Accept the longest draft prefix that matches the verifier.
6. Emit the accepted prefix and the verifier correction token.
7. Slice the decoder cache to the verified prefix.
8. Repeat until the end token or the maximum length.

If step s emits e_s tokens, the average emitted tokens per step is:

$$\bar{e} = \frac{\sum_s e_s}{T},$$

where T is the number of verification steps. A high $\bar{e}$ reduces the number of steps, but it does not guarantee higher throughput because verification also costs time.

3.8 Training Procedure

LLM2Seq uses three phases. Phase 1 freezes the encoder and trains the adapter, decoder, global memory tokens, and LM head with $\mathcal{L}_{sum}$. Phase 2 adds LoRA to the encoder and continues optimizing the summarization loss. Phase 3 freezes the main summarizer and trains only the MTP module with $\mathcal{L}_{MTP}$. This ordering matters. The summarizer is built first. The acceleration module is trained after that, so it does not change the main summarization path. The total trainable parameter set by phase is:

$$\Theta_1 = \{\phi, \psi, E, G_m\},$$

$$\Theta_2 = \Theta_1 \cup \{A_{LoRA}, B_{LoRA}\},$$

$$\Theta_3 = \Theta_{MTP}.$$

3.9 Complexity and Inference Cost

The encoder runs once per source document. The autoregressive decoder then runs once per generated token. Let C_{AR} be the average cost of one autoregressive decoding step. If a summary has n generated tokens, the decoding cost is roughly:

$$C^{AR}_{\text{decode}} \approx nC_{\text{AR}}.$$

Verified MTP emits $\bar{e}$ tokens per step on average, but each step has extra draft and verification cost C_{ver}:

$$C^{MTP}_{\text{decode}} \approx \frac{n}{\bar{e}}(C_{\text{AR}} + C_{\text{ver}}).$$

MTP is faster only when:

$$\bar{e} > 1 + \frac{C_{\text{ver}}}{C_{\text{AR}}}.$$

This explains why accepted tokens and real throughput can disagree. The system must save more decoder steps than it adds in verification overhead.

4 Experimental Setup

4.1 Dataset and Preprocessing

We evaluate on the Vietnamese subset of WikiLingua, a multilingual abstractive summarization dataset extracted from WikiHow articles (Ladhak et al., 2020). Both source articles and summaries are Vietnamese. The original WikiLingua release reports 19,600 Vietnamese article-summary pairs. Our local split contains 19,581 raw records. Preprocessing removes one test example with an empty source, leaving 19,580 examples. Table 2 summarizes the split.

Split	Raw	Used
Train	13,999	13,999
Validation	1,680	1,680
Test	3,902	3,901
Total	19,581	19,580

Table 2: Vietnamese WikiLingua split used in this report.

4.2 Compared Systems and Baselines

We compare three systems. **LLM2Seq-Llama** uses the LLM2Vec-Sheared-LLaMA encoder. **LLM2Seq-Qwen** uses the Qwen3-Embedding-0.6B encoder. Both use the same adapter and lightweight decoder design. **T5Gemma** is a T5Gemma-2-1B-1B LoRA baseline (Zhang et al., 2025).

4.3 Implementation Details

The source length is up to 3,072 tokens and the target length is up to 384 tokens during training. Generation uses at most 256 new tokens. All reported systems use greedy decoding and no-repeat trigram blocking. LLM2Seq uses bf16/tf32 training. Both LLM2Seq variants adapt the encoder with LoRA. The MTP module uses four heads and is trained after the main summarizer is frozen. We use AdamW with betas $(0.9, 0.95)$ and $\epsilon = 10^{-8}$, a linear warmup, cosine decay, and gradient clipping at 1.0. Phase 1 trains the adapter and decoder for 6 epochs with batch size 24, gradient accumulation 2, learning rate $2 \cdot 10^{-4}$, warmup ratio 0.03, and weight decay 0.01. Phase 2 adapts the encoder with LoRA. For Llama, Phase 2 runs for 4 epochs with batch size 16, gradient accumulation 2, encoder learning rate 10^{-4}, decoder and adapter learning rate $5 \cdot 10^{-5}$, and weight decay 0.01. For Qwen, Phase 2 runs for 6 epochs with batch size 8, gradient accumulation 4, encoder learning rate $5 \cdot 10^{-4}$, decoder and adapter learning rate $5 \cdot 10^{-5}$, and weight decay 0.05. The Qwen run also uses gradual unfreezing. Phase 3 freezes the encoder, adapter, and decoder, then trains only the MTP heads for 6 epochs with batch size 20, gradient accumulation 4, learning rate $5 \cdot 10^{-4}$, warmup ratio 0.05, and weight decay 0.01. Checkpoints are selected by validation loss. For Llama and T5Gemma, generation uses a minimum of 32 new tokens and repetition penalty 1.15. For the Qwen comparison log, generation uses a minimum of 64 new tokens and repetition penalty 1.05. T5Gemma is trained for 4 epochs with LoRA, batch size 16, gradient accumulation 2, learning rate 10^{-4}, warmup ratio 0.03, weight decay 0.01, cosine schedule, and fused AdamW. Appendix A summarizes the training and decoding configuration.

4.4 Evaluation Metrics

We report ROUGE (Lin, 2004), length statistics, and speed. ROUGE-1 and ROUGE-2 use unigram and bigram overlap. Let $\hat{y}$ be the generated summary and y be the reference. For $n \in \{1, 2\}$, let $C_n(\hat{y}, y)$ be the clipped count of overlapping n-grams:

$$P_n = \frac{C_n(\hat{y}, y)}{\#n\text{-grams}(\hat{y})}, \qquad R_n = \frac{C_n(\hat{y}, y)}{\#n\text{-grams}(y)}.$$

The reported ROUGE-n is F1:

$$\text{ROUGE-}n = \frac{2P_nR_n}{P_n + R_n}.$$

ROUGE-L uses the longest common subsequence:

$$P_L = \frac{\text{LCS}(\hat{y}, y)}{|\hat{y}|}, \qquad R_L = \frac{\text{LCS}(\hat{y}, y)}{|y|},$$

$$\text{ROUGE-}L = \frac{2P_LR_L}{P_L + R_L}.$$

For length, we report mean predicted words and too-long rate. For Vietnamese text, ROUGE is computed with the Python `rouge-score` implementation using `use_stemmer=False`. Decoded strings are evaluated directly. Length statistics use whitespace-separated tokens from `text.split()`, which we call words for readability. We do not apply Vietnamese word segmentation such as VnCoreNLP or underthesea. This choice is reported explicitly because Vietnamese word segmentation can affect n-gram

Model	R-1	R-2	R-L	Pred.	Long
Llama	48.36	15.54	29.05	29.10	5.46
Qwen	54.01	20.83	31.83	58.31	35.79
T5Gemma	**54.24**	**27.42**	**33.89**	90.89	68.03

Table 3: WikiLingua test results. ROUGE values are F1 percentages. Pred. is mean predicted words. Long is too-long rate in percent. Reference summaries average 51.86 words.

overlap and length statistics. For speed, we report generated tokens per second and mean latency. We compare speed only when the batch setting is the same. The AR-vs-MTP speedups are taken from paired comparison logs within each LLM2Seq encoder.

4.5 Research Questions

RQ1: Can pretrained LLM-based source encoders be reused inside an encoder-decoder summarizer? **RQ2**: How much do the LLM2Seq encoder configurations affect quality, length, and speed? **RQ3**: Does verified MTP improve real throughput, or does it only reduce decoding steps?

5 Results and Analysis

5.1 Main Summarization Results

Table 3 reports the main test results. T5Gemma has the best ROUGE-2 and ROUGE-L. LLM2Seq-Qwen is close on ROUGE-1 and much stronger than LLM2Seq-Llama. This provides evidence that LLM-based source encoders can support encoder-decoder summarization under the Qwen-based LLM2Seq setting. Because the two LLM2Seq variants also differ in training schedule and decoding constraints, this is a system-level comparison rather than an isolated encoder ablation.

5.2 Length and Conciseness Analysis

The Llama variant under-generates. Its mean output is 29.10 words, far below the 51.86-word reference mean. Qwen is closer to the reference length, with 58.31 words. T5Gemma reaches the best ROUGE scores, but it generates much longer summaries, with 90.89 words on average and a 68.03% too-long rate. Thus, T5Gemma is stronger by ROUGE, while LLM2Seq-Qwen is more concise.

Model	tok/s	Mean latency
LLM2Seq-Llama	114.74	0.697s
LLM2Seq-Qwen	95.87	0.779s

Table 4: Autoregressive decoding speed for LLM2Seq variants on WikiLingua under the same batch setting.

5.3 Autoregressive Inference Speed

Table 4 reports autoregressive decoding speed for the two LLM2Seq variants. These runs share the same evaluation setting. T5Gemma speed is omitted because the available T5Gemma inference log uses a different batch setting.

5.4 MTP Analysis: Accepted Tokens vs Throughput

Table 5 gives the main MTP result. MTP increases tokens per decoding step for both encoders. For Llama, this does not become a speedup. It emits 2.45 tokens per step, but throughput drops to 0.64x. For Qwen, MTP emits 2.30 tokens per step and reaches a real 1.17x speedup.

The key result is not the accepted-token or emitted-token count alone. It is the gap between tokens per step and real throughput. Verifier cost, tensor control flow, and implementation overhead decide whether MTP helps. Although verified MTP is designed to follow the constrained greedy path, the current implementation can still show small metric differences because the AR and MTP evaluation paths may differ in cache handling, stopping behavior, or decoding constraints. We therefore report the quality scores for both modes instead of assuming exact output equivalence. This answers RQ3.

5.5 Comparison with End-to-End Context Compression

We do not claim a direct empirical comparison with LCLMs. The current project does not include an LCLM baseline. The comparison is conceptual. As summarized in Table 1, LCLMs compress contexts into shorter latent sequences for general long-context inference (Li et al., 2026). LLM2Seq keeps token-level source memory and trains for Vietnamese summarization. This distinction matters because LLM2Seq does not optimize compression ratio; instead, it studies whether token-level LLM representations can serve as effective source memory for summarization. A fair future comparison should add block compression base-

Model	Mode	tok/s	Spd.	Lat.	Emit	Draft	R-1	R-2	R-L
Llama	AR	114.74	1.00x	0.697	1.00	0.00	48.36	15.54	29.05
Llama	MTP	73.14	0.64x	1.094	2.45	0.53	48.36	15.54	29.05
Qwen	AR	95.87	1.00x	0.779	1.00	0.00	54.01	20.83	31.83
Qwen	MTP	**111.71**	**1.17x**	**0.662**	2.30	0.44	53.90	20.81	31.64

Table 5: Self-distilled verified MTP results. AR means autoregressive decoding. Lat. is mean latency in seconds. Emit is emitted tokens per decoding step. Draft is accepted draft tokens per step, excluding the main token and verifier correction.

lines, such as mean-pooling encoder states by 4, 8, or 16 tokens before the decoder.

5.6 Qualitative Example

Appendix C gives examples from the test prediction files. It shows the same pattern as the aggregate results. LLM2Seq-Llama is short but loses key content. LLM2Seq-Qwen often keeps the broad topic and a concise style, but it can also make serious semantic errors. T5Gemma is more fluent and covers more details, but it is also longer. This example supports the main reading of Table 3: different LLM2Seq configurations lead to different content-selection and length behaviors.

5.7 Error Analysis

The results reveal three error patterns. First, LLM2Seq-Llama under-generates, which hurts recall and leads to lower ROUGE. Second, T5Gemma often generates much longer summaries. This can improve overlap scores but reduces conciseness. Third, verified MTP can preserve quality while still losing speed if the verifier overhead dominates. The Llama MTP result is the clearest example of this failure mode.

6 System Demonstration

6.1 Web Interface

We implement a web interface for running LLM2Seq summarization. The interface loads the Qwen-based model, accepts a source document, and shows the generated summary. It is designed for interactive testing rather than for reporting final quantitative claims.

6.2 Standard Autoregressive Decoding Mode

In autoregressive mode, the model generates one token at a time with the main decoder. This mode is the reference behavior for the summarizer.

6.3 Verified MTP Decoding Mode

In verified MTP mode, the MTP heads draft future tokens and the frozen decoder verifies them. The summary remains constrained by the main decoder, while the system may reduce the number of decoding steps.

6.4 Reported Runtime Statistics

After generation, the demo reports latency, generated tokens, and tokens per second. In MTP mode, it also reports accepted-token statistics and the observed speedup for that run.

6.5 Scope of the Demonstration

Appendix D shows the same WikiLingua example under the two decoding modes. The demo checks the end-to-end system. It is not the main evidence for quality or speed. All quantitative claims come from the controlled evaluation logs.

7 Discussion

7.1 What LLM2Seq Demonstrates

LLM2Seq shows that LLM-based source encoders can be reused inside an encoder-decoder summarizer. The Qwen variant is close to T5Gemma on ROUGE-1 while producing shorter summaries. This supports the core idea, but it does not prove that LLM2Seq dominates native encoder-decoder models.

7.2 Why Source Encoder Configuration Matters

The Qwen-based LLM2Seq variant achieves higher ROUGE than the Llama-based variant under our tuned training and decoding settings. The decoder and adapter design are the same. This suggests that source encoder configuration is likely a major practical factor, although the comparison is not an isolated encoder-only ablation. The adapter can only expose information that exists in the encoder states.

7.3 Why MTP Speedup Is Limited

For Qwen, MTP gives 2.30 tokens per step but only 1.17x real speedup. For Llama, the overhead dominates and throughput drops. This matches the cost condition in Section 3. MTP helps only when accepted tokens pay for verification.

7.4 When Token-Level Memory Is Better Than Compression

Token-level memory is useful when the summary depends on local source evidence. This is common in how-to articles, where small details can change the correct summary. Compression may help when the source is very long and redundant. The current results do not settle this trade-off. They show that token-level memory is a reasonable design point for WikiLingua summarization.

7.5 Practical Deployment Considerations

The current verified decoder is an implementation-level prototype. Future deployment should fuse verification, reduce Python control flow, and use optimized runtimes such as vLLM. Quantization methods such as SmoothQuant may also help under memory limits (Xiao et al., 2023).

8 Threats to Validity

8.1 Dataset Validity

The experiments use only the Vietnamese subset of WikiLingua. Results may differ on news, dialogue, scientific, or legal summarization.

8.2 Baseline Validity

T5Gemma is a strong baseline, but its available speed log uses a different batch setting from LLM2Seq. We therefore compare T5Gemma for quality and length, not for speed.

8.3 Metric Validity

ROUGE measures lexical overlap. It does not fully measure factual consistency, fluency, or usefulness. Length differences can also affect overlap scores. For Vietnamese, whitespace tokenization is another source of variation because one semantic word can contain multiple whitespace-separated syllables.

8.4 Implementation Validity

The MTP speed result depends on the current implementation. A fused runtime may change the speedup. The Llama negative result should therefore be read as a system result, not as a theoretical limit.

8.5 Generalization Validity

The report studies two encoder configurations and one dataset. More encoders, more languages, and more summarization datasets are needed before making broader claims.

9 Conclusion

LLM2Seq studies how to build an encoder-decoder summarizer from repurposed LLM source encoders. The model uses a gated residual adapter to map source-side LLM states into decoder memory. It then uses a lightweight cross-attentive decoder to generate summaries. On Vietnamese WikiLingua, the Qwen-based variant is competitive with T5Gemma on ROUGE-1 and produces shorter summaries. Self-distilled MTP gives a real 1.17x speedup for Qwen, but the Llama result shows that fewer decoding steps are not enough. Future work should add adapter ablations, add compression baselines, and optimize verified decoding.

Limitations

This report does not include a full adapter ablation or an LCLM-style block-compression baseline. It also does not include human factuality judgments. The current demo is an engineering artifact for inspection, not evidence for user-facing reliability.

Ethics Statement

This work uses an existing public summarization dataset and does not introduce human-subject data collection. The system may still produce incomplete or incorrect summaries. It should not be used as the only source of information in high-stakes settings.

Broader Impact

LLM2Seq may make summarization systems easier to build from existing LLM checkpoints. This can reduce training cost and make summarization more accessible for lower-resource languages. At the same time, better summarization tools can also spread errors faster if users over-trust generated summaries.

Code Availability

The implementation, evaluation scripts, verified MTP decoding code, and demo interface are available at: https://github.com/BienKieu1411/LLM2Seq.

Acknowledgments

We thank the project reviewers for feedback on framing, experimental reporting, and report structure.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Faisal Ladhak, Esin Durmus, Claire Cardie, and Kathleen McKeown. 2020. WikiLingua: A new benchmark dataset for cross-lingual abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4034–4048, Online. Association for Computational Linguistics. Dataset release: https://github.com/esdurmus/Wikilingua.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880.

Ang Li, Sean McLeish, Haozhe Chen, Nimit Kalra, Zaiqian Chen, Artem Gazizov, Venkata Anoop Suhas Kumar Morisetty, Bhavya Kailkhura, Harshitha Menon, Zhuang Liu, Brian R. Bartoldson, Tom Goldstein, Sanae Lotfi, Micah Goldblum, and Pavel Izmailov. 2026. End-to-end context compression at scale. *arXiv preprint arXiv:2606.09659*.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.

Yingfeng Luo, Tong Zheng, Yongyu Mu, Bei Li, Qinghong Zhang, Yongqi Gao, Ziqiang Xu, Peinan Feng, Xiaoqian Liu, Tong Xiao, and Jingbo Zhu. 2025. Beyond decoder-only: Large language models can be good encoders for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9399–9431.

Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. Generative representational instruction tuning. *arXiv preprint arXiv:2402.09906*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, and 1 others. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Biao Zhang, Paul Suganthan, Gael Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar Sanseviero, Shreya Pathak, Cassidy Hardin, Francesco Visin, Jiageng Zhang, Kathleen Kenealy, Qin Yin, Olivier Lacombe, Armand Joulin, Tris Warkentin, and Adam Roberts. 2025. T5Gemma 2: Seeing, reading, and understanding longer. *arXiv preprint arXiv:2512.14856*.

Guoliang Zhao, Ruobing Xie, An Wang, Shuaipeng Li, Huaibing Xie, and Xingwu Sun. 2026. Self-distillation for multi-token prediction. *arXiv preprint arXiv:2603.23911*.

A Summary of Hyperparameters

The main hyperparameters are:

- Source and target length: 3,072 source tokens; 384 target tokens.
- Generation: greedy decoding; at most 256 new tokens; no-repeat trigram blocking.
- Adapter: selected layers $[-1, -4, -8, -12]$; salience gate; 4-layer EncStack; 16 global memory tokens.
- Decoder: 8 layers; hidden size 1024; 16 heads; FFN size 4096; dropout 0.1.
- LoRA: encoder adapters are used during Phase 2.
- MTP: 4 heads; CE weight 0.6; KL weight 0.2; head weights $[1.0, 0.8, 0.5, 0.25]$.

B Additional Experimental Results

The current project snapshot includes the main quality, length, speed, and MTP results reported in this report. It does not include a complete adapter ablation. For that reason, this report does not include an ablation table.

C Additional Qualitative Examples

C.1 Representative Case

The following example comes from test item `test_003594`. The source is a Vietnamese WikiHow-style article about skin care. We show a source excerpt for space.

Source (excerpt). Sau khi kết thúc một ngày, trang điểm, bụi bẩn, dầu nhờn bám đầy trên mặt. Hãy tạo thói quen làm sạch da trước khi đi ngủ. Dùng dầu tẩy trang. Nó không chỉ có tác dụng làm thông thoáng lỗ chân lông, mà còn ngăn vi khuẩn trên gối xâm nhập vào da khi ngủ. Dùng xà phòng dịu nhẹ.

Reference. Rửa mặt vào buổi tối. Dùng nước hoa hồng. Dùng kem dưỡng ẩm. Tẩy tế bào chết một tuần một lần. Bảo vệ da khỏi ánh nắng mặt trời.

LLM2Seq-Qwen. Rửa mặt hai lần một ngày. Dùng nước cân bằng da (toner). Tẩy tế bào chết cho da. Dưỡng ẩm cho da sau khi rửa mặt. Dành thời gian chăm sóc da sau mỗi bên ngoài. Tự pha loãng một cách tự nhiên.

T5Gemma. Rửa mặt trước khi đi ngủ. Dùng nước hoa hồng để làm se lỗ chân lông và loại bỏ dầu thừa trên da. Sử dụng sữa dưỡng ẩm vào ban ngày hoặc đêm, tùy theo nhu cầu của bạn. Tẩy tế bào chết mỗi tuần một lần nếu cần thiết. Bôi kem chống nắng hàng ngày trong suốt mùa hè.

Comment. Both systems recover several main skin-care actions. LLM2Seq-Qwen is concise and keeps core steps, but it still produces disfluent trailing phrases. This makes the example a representative mixed case rather than a clear success. T5Gemma is more complete and fluent, but also longer.

C.2 Failure Case

The following example comes from test item `test_002672`. The source is a Vietnamese WikiHow-style article about crying and emotional expression. We show a source excerpt for space.

Source (excerpt). Bạn đã được dạy rằng những người dũng cảm thì thường không khóc? Những người được dạy cách kiềm chế và giấu kín cảm xúc từ bé thường gặp khó khăn trong việc biểu lộ cảm xúc khi họ lớn lên. Nhưng khóc là một phần thiết yếu của cuộc sống giúp cải thiện tâm lý. Khóc có thể là biểu hiện của nỗi buồn, sự đau đớn, nỗi sợ, hạnh phúc, hay thậm chí là niềm vui.

Reference. Quên hết những gì đã biết về khóc. Hiểu được lợi ích của khóc. Tìm hiểu lý do tại sao bạn phải kiềm chế cảm xúc. Cho phép chính mình khóc cũng là một cách chăm sóc bản thân tốt hơn.

LLM2Seq-Llama. Nhận biết cảm giác tội lỗi và hành động. Xác định những suy nghĩ tiêu cực trong cuộc sống của bạn. Tìm kiếm sự trợ giúp chuyên nghiệp.

LLM2Seq-Qwen. Hãy hiểu rằng cảm xúc của bạn là bình thường. Hãy làm gì đó. Hãy để bản thân bị trầm cảm. Hãy cho phép bản thân cảm nhận sự khóc. Hãy nhớ rằng việc phải là người luôn bộc lộ bản thân. Hãy cười ra khỏi vùng an toàn của bạn. Hãy vui vẻ và đau khổ.

T5Gemma. Khóc là một cách tự nhiên để thể hiện cảm xúc. Khóc không chỉ dành riêng cho trẻ em, mà còn có ích lợi đối với người lớn tuổi như bạn. Cố gắng khóc khi cần thiết và đừng ngại ngùng nếu mình đã bắt đầu rơi nước mắt. Hãy nhớ rằng việc biểu lộ những cảm xúc của bản thân sẽ giúp cải thiện tâm trạng. Thử tưởng tượng về thời điểm trước đây khi bạn bật khóc vì điều gì đó.

Comment. This example shows that LLM2Seq-Qwen can preserve conciseness but may produce semantically inappropriate phrases. This motivates future factuality and human evaluation. It also shows why automatic overlap and length statistics must be read together with output inspection.

D Demo Interface Details

The demo provides a source text box, a decoding-mode selector, and a maximum-token control. It reports the generated summary, latency, generated tokens, and tokens per second. In verified MTP mode it additionally reports accepted-token statistics.

E Architecture Details

The adapter uses token-wise layer fusion, a gated residual projection, token salience gating, Enc-Stack memory refinement, and learned global memory tokens. The decoder attends to the final memory M through cross-attention in every layer.

F Verified MTP Decoding Pseudocode

1. Encode the source once and build adapted memory M.
2. Start with the current target prefix.
3. Predict the main next token with the frozen decoder.
4. Draft future tokens with the cascaded MTP heads.
5. Verify the candidate block with the frozen main decoder.
6. Accept the longest draft prefix that matches the verifier.
7. Append the accepted prefix and the verifier correction token.
8. Slice the decoder cache to the verified prefix.
9. Stop when the end token or maximum length is reached.

G Dataset Preprocessing Details

WikiLingua examples contain source sentence lists and target sentence lists. Preprocessing joins source sentences into one source document and joins target sentences into one summary paragraph. One test record with an empty source is removed.

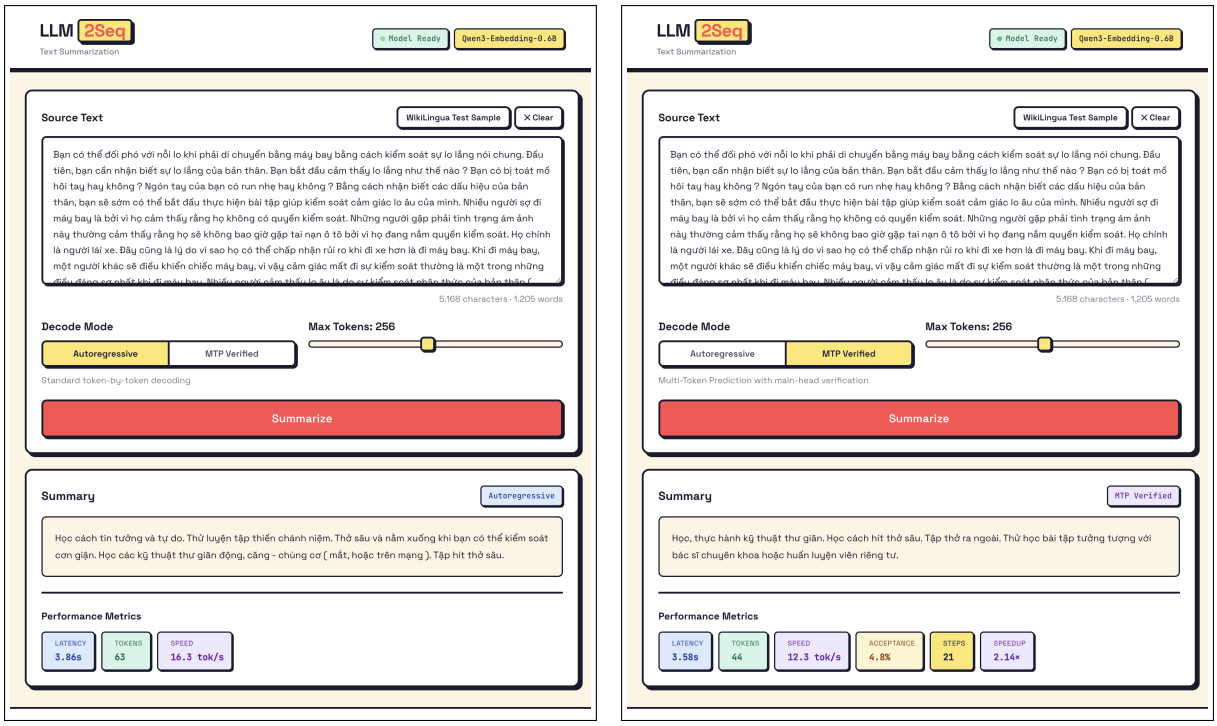

Figure 2: LLM2Seq web demo on the same WikiLingua sample. Left: standard autoregressive decoding. Right: verified MTP decoding.